

DAT240 - Report for Campus Eats

Navn	Studentnummer
Abdirahman Abdi Ibrahim	283989
Abdulaziz Abdulkadir Hassan	258884
Daniil Khanchych	280886
William Ditlev Hanssen	283984
Jonas Sørvik-Jacobsen	283981

Contents

1	Introduction	2
2	Team Organization and Workflow	2
3	Technical decisions and their alignment with the Design Document	3
3.1	RabbitMQ	4
3.2	Smtplib	4
3.3	SignalR	4
4	Challenges and Problem Solving	4
4.1	Advanced requirements difficulties	5
4.2	Testing issues	5
4.3	Common problems	5
4.4	Problem solving	5
4.5	Final reflection	6
5	Group Experience	6
5.1	Issues encountered	6
5.2	Midway issues	6
5.3	Changes to process	6
6	Conclusion	7

1 Introduction

For our group project, we selected Campus Eats. This proved to be a good candidate for practicing domain driven design(DDD), offering a well defined structure with clearly separable components such as menu handling, and order processing. In this document, we outline our team organization and workflow, discuss the technical decisions made throughout development and how they align with our design document, reflect on the challenges we encountered and how we resolved them, and share our overall group experience, including the issues we faced, how they evolved over time, and the adjustments we made to our process along the way.

2 Team Organization and Workflow

We organized our work into pairs, where one person did the coding, while the other gave input. We also made a discord server for our group, where we gave each other updates on what has been done as well as share new updates and other information. On the server we have used Apollo discord bot to post work sessions for us. With this we can see who will attend the session, when it will be, and what we will be working on. Before starting writing code we created overview of the project 1. We created a plan at the start so we could focus our work.

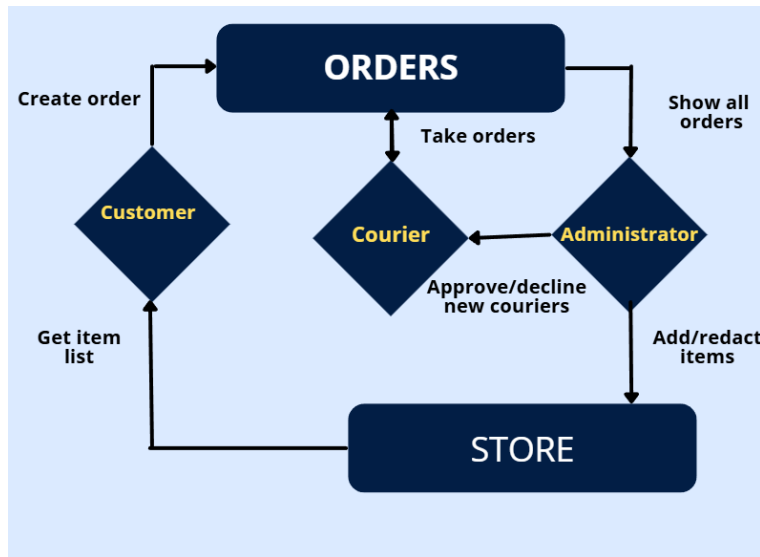


Figure 1: One of the early diagrams that were used to plan project structure

We organized the project into 3 parts, first worked hard on backend, then frontend and at last advanced requirements. This separation of workflow made

it possible to create a solid backbone that could expand nicely. By continuously creating tests and having a good CI pipeline, version control and merging became easy.

3 Technical decisions and their alignment with the Design Document

When it comes to technical decisions, we chose to use C# for the backend, Blazor Pages for the frontend, and Tailwind CSS for styling. These choices align well with what was outlined in our design document. C# was selected for the backend to gain experience with a widely used industry language and to develop a project that would strengthen our resumes. Blazor Pages were chosen for the frontend based on a recommendation from the lecturer, although our original plan was to use Razor Pages.

After finishing the backend, we started working on the design of the frontend. To get a better overview, we drew some sketches of how the project should look like. We created a sketchbook that can be found in the GitHub repository, but here is an example of the sketch.

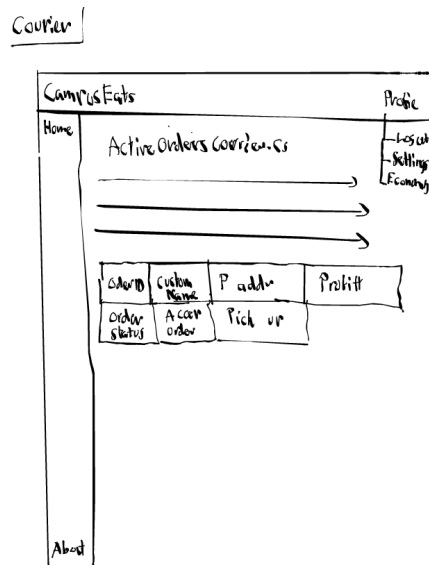


Figure 2: Courier Dashboard frontend plan

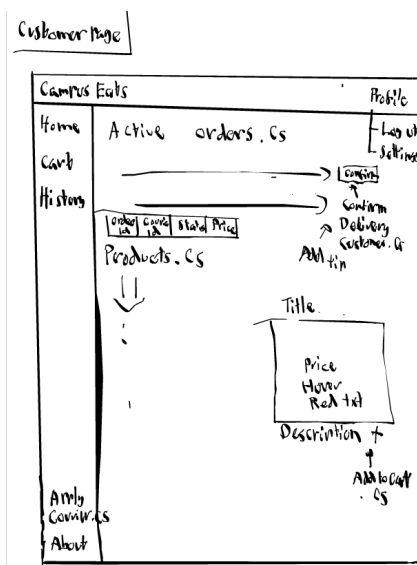


Figure 3: Customer Dashboard frontend plan

Additionally, we have used libraries and technologies such as MediatR, SignalR, RabbitMQ, and SMTP. MediatR was used for implementing the mediator pattern, which helped decouple components by routing requests through centralized

handlers. This improves the structure of the application by separating business logic from controllers and increasing maintainability.

3.1 RabbitMQ

RabbitMQ was used as a message broker to support asynchronous communication between services. By publishing messages to a queue instead of handling tasks synchronously, we were able to decouple services and improve scalability and responsiveness. This was particularly useful for functionality such as email handling, where immediate responses were not required. Using RabbitMQ also made the system more modular, as producers and consumers could evolve independently without tight coupling.

3.2 Smtplib4dev

Smtplib4dev was used as a local development tool to test email functionality without sending real emails. It provided a simple SMTP server and a web-based interface where sent emails could be inspected, making debugging and verification significantly easier during development. By using Smtplib4dev, we were able to safely test email related features in isolation and ensure correct behavior before deployment.

3.3 SignalR

SignalR was used to enable real-time push notifications to the browser. On its own MediatR only handles communication inside the server and has no way of reaching a client. By combining them MediatR handles the internal flow when an order status changes while SignalR delivers that update to the correct user's browser instantly. This allowed us to notify customers and couriers in real time without requiring any page refresh or polling.

4 Challenges and Problem Solving

One of the most significant challenges encountered during development was the introduction of new technical requirements at a relatively late stage in the project. During the initial planning phase, we chose not to fully explore the advanced or optional requirements, focusing instead on establishing a working core system. While this allowed us to make early progress, it later proved to be a weakness, as several advanced requirements introduced substantial technical complexity. A clear example of this was the implementation of the email service, which unexpectedly required the integration of the RabbitMQ library. Since this dependency was not anticipated early on, we had to adapt our architecture and learning process under time pressure, making the task significantly more demanding than if it had been planned from the start.

4.1 Advanced requirements difficulties

Similarly to the email service, push notifications required careful planning and the integration of SignalR to support real-time updates, which introduced an even greater degree of code complexity. Each advanced requirement was implemented in isolation within its own dedicated branch. While this approach helped manage complexity during development, the subsequent merging process presented its own challenges. In particular, naming conflicts arose between the email handling and push-notification handling components, requiring careful review and manual resolution during merge.

4.2 Testing issues

Testing was another major challenge, particularly in the early stages of the project. None of the group members had extensive experience designing tests from scratch, which resulted in initial uncertainty regarding both tools and best practices. After some experimentation, we decided to use a database context together with NUnit for testing. This decision turned out to be effective, as NUnit supported not only unit tests but also integration tests, which became increasingly important during later stages of development. As the system grew more complex, these integration tests helped us catch errors that would have been difficult to detect otherwise.

4.3 Common problems

A recurring technical issue early in development involved concurrency problems related to testing. These issues were partly caused by an incorrect initial project setup, which ultimately led us to abandon our original solution and restart the project under a new structure. This is reflected in the repository name, `CampusEatsv2`, indicating that the first version of the project was fundamentally flawed in its configuration. Although restarting was a frustrating decision at the time, it significantly improved stability and clarity moving forward.

4.4 Problem solving

Once the foundational setup was corrected, we attempted to divide the system into individual services and assign each service to a pair of developers. However, this approach initially struggled due to a lack of shared conventions and uncertainty about how services should be structured, tested, and integrated. Recognizing this issue, we dedicated a full meeting to defining a common setup for all services, including shared testing strategies and project structure. This proved to be a turning point in the project, as it reduced ambiguity, improved collaboration between pairs, and made parallel development much more efficient.

4.5 Final reflection

Overall, the challenges we faced highlighted the importance of early technical planning, shared standards, and willingness to re-evaluate decisions when problems arise. While several issues could have been mitigated with better foresight, the solutions we implemented strengthened the project and improved both the quality of the final system and our understanding of collaborative software development.

5 Group Experience

Throughout the project, our primary goal was to work collaboratively in person, using meetings as the main forum for discussion, decision-making, and implementation. This approach worked well once established, as it allowed us to quickly identify misunderstandings and resolve issues together. However, early in the process we underestimated how much communication was required to keep everyone aligned, which led to some initial friction within the group.

5.1 Issues encountered

One challenge we encountered was differing work habits. While some members preferred to complete tasks independently between meetings, this sometimes resulted in undocumented changes or code being produced without sufficient explanation to the rest of the group. This created moments of confusion and slowed integration, particularly when transitioning between systems or handing off components. Through reflection, we realized that effective communication was not only about meeting frequently, but also about ensuring transparency and shared understanding between meetings.

5.2 Midway issues

Through the project, we deliberately paused to evaluate our process and communication practices. During a dedicated feedback meeting, we openly discussed what was not working, including insufficient documentation, unclear task division, and unrealistic expectations regarding availability. This reflection led us to refine our workflow by clarifying responsibilities, setting clearer expectations for documentation, and agreeing on communication standards for work done outside scheduled meetings.

5.3 Changes to process

The changes we implemented proved effective. Planning became more structured, collaboration improved, and dividing tasks became significantly easier. Although working with individuals who had different starting points and working styles remained somewhat challenging, the increased focus on communication reduced friction overall. This experience highlighted the importance of

regularly reflecting on group processes, being willing to adapt, and actively maintaining clear communication to ensure effective teamwork.

6 Conclusion

Looking at the entire process of making this application. We have learnt that making a solid plan and setting the requirements for everything that you make is very important for development. Working as a team is difficult, but can be better if you spend time trying to workout what is working and not during the process. Making a common understanding and quality requirements for code implementation can make your life a whole lot easier.